

Broadcast Alerts

Lost Pet Alert System — Broadcast Sub-Flow

PawGuard Platform | pawguard-backend (FastAPI + PostgreSQL + Redis + ARQ) | Prepared for Team Review

1. Purpose

Once a lost-pet report exists, the reporter can trigger a broadcast that pushes the alert to relevant users (e.g. nearby app users / clinic network) instead of the report just sitting passively in search results.

2. Endpoint

Method & Path	Permission	Rate Limit
POST /lost-found/lost/{report_id}/broadcast	lost_found:broadcast	3 requests / hour per user

The strict 3/hour limit exists because broadcast is a push-notification / alert-fan-out action — it must be hard to spam.

3. Flow



1. Service validates the caller is either the original reporter or an admin (`queue_lost_alert_broadcast`).
2. Service validates the report status is still ACTIVE — a resolved/expired report cannot be re-broadcast.
3. A background job “`broadcast_lost_pet_alert`” is enqueued on the ARQ/Redis job queue — the API call itself returns immediately (`queued: true`), keeping the request fast.
4. The ARQ worker picks up the job asynchronously and performs the actual fan-out (in-app notifications / email to the notification service).
5. An audit-log entry (`LOST_FOUND_BROADCAST_QUEUED`) is written with the actor, IP address, and report ID for traceability.

4. Why Asynchronous (Queue-Based)

- Fan-out to many recipients can be slow — doing it inline would block the HTTP response.
- ARQ retries failed deliveries automatically without the user needing to resubmit.
- Keeps the public-facing API responsive even under load.

5. Failure Handling

If the ARQ connection is unavailable, the service fails fast with a clear “Alert delivery is temporarily unavailable” error rather than silently dropping the broadcast.

Source: `lost_found/service.py (queue_lost_alert_broadcast)`, `lost_found/router.py`, `workers/arq_worker.py`.